

Unique Solutions of Guarded Recursive Equations

Rob van Glabbeek* 

School of Informatics, University of Edinburgh, UK

School of Computer Science and Engineering, University of New South Wales, Sydney, Australia

`rvg@cs.stanford.edu`

This paper shows that guarded systems of recursive equations have unique solutions up to strong bisimilarity for any process algebra with a structural operation semantics in the ready simulation format. A similar result holds for simulation equivalence, for ready simulation equivalence and for the (ready) simulation preorder.

1 Introduction

In many process algebras, including CCS [22], CSP [8] and ACP [2], infinite processes are specified by systems $\mathcal{S}$ of recursive equations $\{X = \mathcal{S}_X \mid X \in V_{\mathcal{S}}\}$. Such systems are called *recursive specifications*. Here $V_{\mathcal{S}}$ is a set of recursion variables specific to a recursive specification, and each $X \in V_{\mathcal{S}}$ is defined to behave as the expression $\mathcal{S}_X$. Crucially, X itself, and other variables from $V_{\mathcal{S}}$, may occur in the expression $\mathcal{S}_X$. An important tool in equational reasoning is that a large class of recursive specifications have unique solutions. Here a *solution* is a tuple $\vec{P}$ of processes, one for each variable X in the set $V_{\mathcal{S}}$, such that filling in the X -component of $\vec{P}$ for X , at both the left- and the right-hand side of the recursive equations in $\mathcal{S}$, yields a set of semantically valid statements. Unfortunately, not all recursive specifications have unique solutions. The recursive equation $X = X$, for instance, allows any process as a solution. Milner [22] formulated a syntactic criterion on recursive specifications called *guardedness*, and showed that in the process algebra CCS, systems of guarded recursive equations have unique solutions up to strong bisimilarity. In the setting of ACP [1, 2] two reasoning principles are defined: the *Recursive Definition Principle* (RDP) states any recursive specification has a solution, and the *Recursive Specification Principle* (RSP) states that guarded recursive specifications have at most one solution, up to a semantic equivalence. Whether these principles hold may depend on the process algebra and on the semantic equivalence employed. First of all it takes a semantic equivalence to determine whether a given tuple $\vec{P}$ is a solution of a recursive specification $\mathcal{S}$. Secondly, if $\mathcal{S}$ is guarded and $\vec{P}$ and $\vec{Q}$ both are solutions, RSP demands that $\vec{P}$ and $\vec{Q}$ are semantically equivalent. RDP and RSP are equational and conditional equational laws that can be added as axioms to the equational theories of various process algebras. Doing so has been a crucial step in achieving complete axiomatisations [21, 3, 9, 17].

Subtly different definitions of guardedness appear in the literature. In [22] it was required that within the bodies $\mathcal{S}_X$ of recursive equations the recursion variables $Y \in V_{\mathcal{S}}$ may appear only within the scope of a CCS action prefixing operator $a.$. Here this action *guards* the recursion variable. In other process algebras one might allow other operators as guards. In ACP [2] for instance, the left merge $\parallel$ is a good candidate. However, only the second argument of the left merge guards: the recursive specification $X = b \parallel X$ is guarded, but $Y = Y \parallel b$ is not. In general, a suitable definition of guardedness is whatever syntactic criterion allows one to obtain RSP for a suitable semantic equivalence. Proofs of RSP [22, 1] tend to depend to a large extent on the syntax and semantics of the process algebra involved.

*Supported by Royal Society Wolfson Fellowship RSWF\R1\221008

The goal of this paper is to give a proof of RSP for strong bisimulation semantics for a large class of process algebras, including CCS and ACP, at once. I do this by characterising a format for transition system specifications in structural operational semantics, proposing the most general definition of guardedness I can find in that context, and formally show that all process algebras whose operational semantics fits that format satisfy RSP, for strong bisimilarity. They also satisfy RDP, but that is fairly trivial.

The most general format for standard process algebras (without probabilities, time, name-binding or other special features) that guarantees strong bisimilarity to be congruence is the ntyft/ntyxt format of Groote [18], augmented with recursion as in [16]. Let f (pronounced “after a ”) be a unary operator defined through the transition rules

$$\frac{x \xrightarrow{a} y \quad y \xrightarrow{b} z}{f(x) \xrightarrow{b} z}$$

for the specific action a and each action b . It has lookahead, in the sense that the initial actions of f are determined by looking ahead at the second action performed by its argument. Now the recursive equation $X = f(a.X)$ has any process as solution, even though the recursion variable X appears guarded (by the guard a) in the body $f(a.X)$ of the recursion equation. Because of this I rule out lookahead in this paper. Aiming for maximal generality, I will obtain RSP for strong bisimilarity in the ready simulation format [13], which is the ntyft/ntyxt format without lookahead, augmented with recursion.

Most applications of this work will fall in the GSOS format, which restricts the ready simulation format mainly by allowing only variables to occur in premises, except in the rules for recursion. In this context I favour a notion of guardedness due to Vaandrager [26]: the i^{th} argument of an n -ary operator f counts as a guard iff there is no transition rule of the form

$$\frac{H}{f(x_1, \dots, x_n) \xrightarrow{a} E}$$

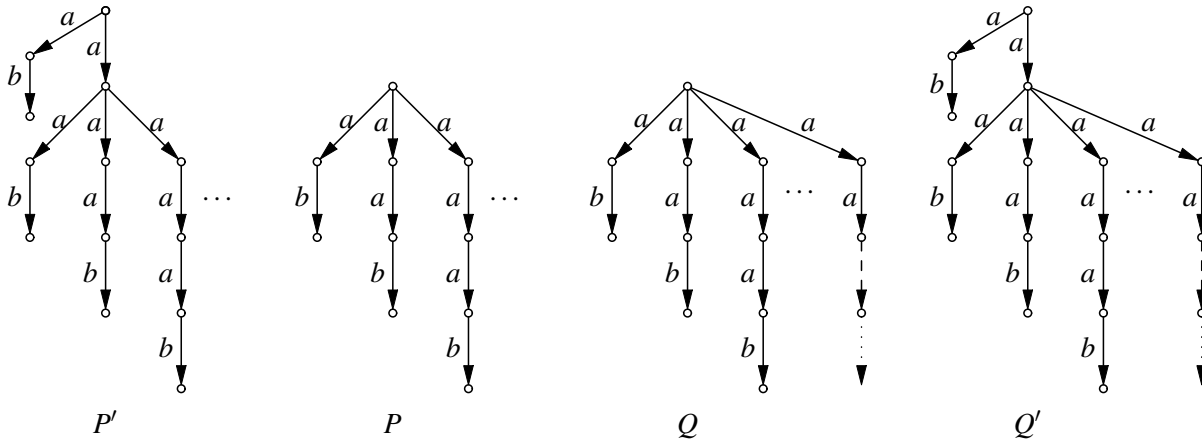
such that the variable x_i occurs as the left-hand side of one of the premises in H . I will generalise this idea to the ready simulation format by allowing x_i to occur in the left-hand sides of premises (which may be complex terms) but only in guarded positions.

After establishing RSP for strong bisimilarity, I observe that by means of an almost identical proof it can also be shown for other strong semantic equivalence at the branching time side of the linear time – branching time spectrum [14], namely simulation equivalence (in the absence of negative premises) and ready simulation equivalence. Naturally I would like to show it for preorders as well. Using the notation $\vec{P} \equiv \mathcal{S}[\vec{P}]$ to indicate that the tuple $\vec{P}$ is a solution of the recursive specification $\mathcal{S}$ up to the semantic equivalence $\equiv$, there are two ways to replace $\equiv$ by a preorder $\sqsubseteq$. When $\vec{P} \sqsubseteq \mathcal{S}[\vec{P}]$ holds I call $\vec{P}$ a presolution of $\mathcal{S}[\vec{P}]$, and when $\mathcal{S}[\vec{P}] \sqsubseteq \vec{P}$ holds I call it a postsolution. The form of RSP for $\equiv$ simply says that if $\vec{P}$ and $\vec{Q}$ are both solutions of $\mathcal{S}$ then $\vec{P} \equiv \vec{Q}$. It turns out I can conclude nothing from the assumption that $\vec{P}$ and $\vec{Q}$ are both presolutions, or both postsolutions. However, an asymmetric form of RSP holds if $\vec{P}$ is a presolution and $\vec{Q}$ a postsolution. In that case $\vec{P} \sqsubseteq \vec{Q}$, at least when taking for $\sqsubseteq$ the strong simulation preorder or the ready simulation preorder. I thus formulate RSP as

$$\frac{\vec{P} \equiv \mathcal{S}[\vec{P}] \quad \mathcal{S}[\vec{Q}] \equiv \vec{Q}}{\vec{P} \equiv \vec{Q}} \quad \frac{\vec{P} \sqsubseteq \mathcal{S}[\vec{P}] \quad \mathcal{S}[\vec{Q}] \sqsubseteq \vec{Q}}{\vec{P} \sqsubseteq \vec{Q}}.$$

In most cases a semantic equivalence $\equiv$ (say simulation equivalence) arises as the kernel of a preorder, meaning that $P \equiv Q$ iff $P \sqsubseteq Q$ and $Q \sqsubseteq P$. In this setting, RSP for $\equiv$ is a simple consequence of RSP for $\sqsubseteq$. Thus, one merely needs to prove the latter.

For linear time equivalences, a generic counterexample to RSP is given by these processes:



Here $P' = a.b + a.P$ and $Q' = a.b + a.Q$, and in all semantics $\equiv$ of [14] between trace and ready trace equivalence one has $P' \equiv P$ and $Q \equiv Q'$. Thus, the guarded recursive specification $\mathcal{S} = \{X = a.b + a.X\}$ has both P and Q as solutions. Hence for all such semantic equivalences that distinguish between P and Q in models of concurrency where P and Q both exist, RSP fails. This applies to semantics that take infinite traces (or infinite ready traces) into account. A reason to distinguish between P and Q is that P is guaranteed to eventually perform a b -transitions, whereas Q is not. For linear time semantics that do not take infinite traces into account, and identify P and Q , RSP may hold. But that is best established by means of fixed point arguments [25], which is out of scope for this paper.

2 Transition system specifications and their meaning

In this paper Var and A are two sets of *variables* and *actions*. Many concepts that will appear are parameterised by the choice of Var and A , but as here this choice is fixed, a corresponding index is suppressed.

Definition 1 (Terms) A *signature* is a set Σ of operator symbols $f \notin Var$, each equipped with an *arity* $ar(f) \in \mathbb{N}$.¹ The set $\mathbb{T}(\Sigma)$ of *terms with recursion* over a signature Σ is defined inductively by:

- $Var \subseteq \mathbb{T}(\Sigma)$,
- if $f \in \Sigma$ and $E_1, \dots, E_{ar(f)} \in \mathbb{T}(\Sigma)$ then $f(E_1, \dots, E_{ar(f)}) \in \mathbb{T}(\Sigma)$,
- If $V_S \subseteq Var$, $\mathcal{S} : V_S \rightarrow \mathbb{T}(\Sigma)$ and $X \in V_S$, then $\langle X | \mathcal{S} \rangle \in \mathbb{T}(\Sigma)$.

A term $c()$ is abbreviated as c . An term $\langle X | \mathcal{S} \rangle$ as appears in the last clause is a *recursive call*, and the function $\mathcal{S}$ therein is called a *recursive specification*. It is often displayed as $\{X = \mathcal{S}_X \mid X \in V_S\}$. So V_S is the domain of $\mathcal{S}$ and $\mathcal{S}_X$ represents $\mathcal{S}(X)$. Each term $\mathcal{S}_Y$ for $Y \in V_S$ counts as a subterm of $\langle X | \mathcal{S} \rangle$. An occurrence of a variable y in a term E is *free* if it does not occur in a subterm of E of the form $\langle X | \mathcal{S} \rangle$ with $y \in V_S$. Let $var(E)$ denote the set of variables occurring free in a term $E \in \mathbb{T}(\Sigma)$, and let $\mathbb{T}(\Sigma, W)$ be the set of terms E over Σ with $var(E) \subseteq W$. $\mathbb{T}(\Sigma) := \mathbb{T}(\Sigma, \emptyset)$ is set of *closed* terms over Σ , modelling processes.

¹This work generalises seamlessly to operators with infinitely many arguments. Such operators occur, for instance, in [7, Appendix A.2]. Hence one may take $ar(f)$ to be any ordinal. An operator, like the *summation* or *choice* of CCS [22], that actually takes any *set* of arguments, needs to be simulated by a family of operators with a *sequence* of arguments (but yielding the same value upon reshuffling of the arguments), one for each cardinality of this set.

Example 1 Let Σ contain three unary operators $a._$, $b._$ and $d._$, and an infix-written binary operator $\parallel$. Let $X, Y, z \in \text{Var}$. Then $\mathcal{S} = \{X = (a.X) \parallel (b.Y), Y = (d.Y) \parallel (X \parallel z)\}$ is a recursive specification, so $\langle X | \mathcal{S} \rangle \in \mathbb{T}(\Sigma)$. Since $V_{\mathcal{S}} = \{X, Y\}$, the only variable that occurs free in this term is z .

As illustrated here, I often choose upper case letters for bound variables (the ones occurring in a set $V_{\mathcal{S}}$) and lower case ones for variables occurring free; this is a convention only.

A recursive specification $\mathcal{S}$ is meant to denote a $V_{\mathcal{S}}$ -tuple (in the example above a pair) of processes that—when filled in for the variables in $V_{\mathcal{S}}$ —forms a solution to the equations in $\mathcal{S}$.² The term $\langle X | \mathcal{S} \rangle$ denotes the X -component of such a tuple.

Definition 2 (Substitution) A Σ -substitution σ is a partial function from Var to $\mathbb{T}(\Sigma)$; it is *closed* if it is a total function from Var to $\mathbb{T}(\Sigma)$. If σ is a substitution and S any syntactic object, then $S[\sigma]$ denotes the object obtained from S by replacing, for x in the domain of σ , every free occurrence of x in S by $\sigma(x)$, while renaming bound variables if necessary to prevent name-clashes. In that case $S[\sigma]$ is called a *substitution instance* of S . A substitution instance $S[\sigma]$ where σ is given by $\sigma(x_i) = u_i$ for $i \in I$ is denoted as $S[u_i/x_i]_{i \in I}$, and for $\mathcal{S}$ a recursive specification $\langle E | \mathcal{S} \rangle$ abbreviates $E[Y | \mathcal{S}]_{Y \in V_{\mathcal{S}}}$.

Example 2 Extend Σ from Ex. 1 with a constant c . Then

$$\begin{aligned} \langle X | \mathcal{S} \rangle[b.c/z] &= \langle X | \{X = (a.X) \parallel (b.Y), Y = (d.Y) \parallel (X \parallel b.c)\} \rangle, \\ \langle X | \mathcal{S} \rangle[X/z] &= \langle Z | \{Z = (a.Z) \parallel (b.Y), Y = (d.Y) \parallel (Z \parallel X)\} \rangle \text{ and} \\ \langle X | \mathcal{S} \rangle[b.c/Y] &= \langle X | \mathcal{S} \rangle. \end{aligned}$$

Structural operational semantics [23] defines the meaning of system description languages whose syntax is given by a signature Σ . It generates a transition system in which the states, or *processes*, are the closed terms over Σ —representing the remaining system behaviour from that state—and transitions between processes are supplied with labels. The transitions between processes are obtained from a transition system specification, which consists of a set of transition rules.

Definition 3 (Transition system specifications) Let Σ be a signature. A *positive Σ -literal* is an expression $E \xrightarrow{a} E'$ and a *negative Σ -literal* an expression $E \not\xrightarrow{a}$ with $E, E' \in \mathbb{T}(\Sigma)$ and $a \in A$. For $E, E' \in \mathbb{T}(\Sigma)$ the literals $E \xrightarrow{a} E'$ and $E \not\xrightarrow{a}$ are said to *deny* each other. A *transition rule* over Σ is an expression of the form $\frac{H}{\alpha}$ with H a set of Σ -literals (the *premises* or *antecedents* of the rule) and α a positive Σ -literal (the *conclusion*). The terms at the left- and right-hand side of α are the *source* and *target* of the rule. A rule $\frac{H}{\alpha}$ with $H = \emptyset$ is also written α . A literal or transition rule is *closed* if it contains no free variables. A *transition system specification (TSS)* is a pair $(\Sigma, \mathcal{R})$ with Σ a signature and $\mathcal{R}$ a set of transition rules over Σ ; it is *positive* if all antecedents of its rules are positive.

The concept of a (positive) TSS presented above was introduced in GROOTE & VAANDRAGER [19]; the negative premises $E \not\xrightarrow{a}$ were added in GROOTE [18]. The notion generalises the *GSOS rule systems* of [5] and constitutes a formalisation of PLOTKIN's *Structural Operational Semantics (SOS)* [23] that is sufficiently general to cover many of its applications.

The following definition (from [13]) tells when a transition is provable from a TSS. It generalises the standard definition (see e.g. [19]) by (also) allowing the derivation of transition rules. The derivation of a transition $E \xrightarrow{a} E'$ corresponds to the derivation of the transition rule $\frac{H}{E \xrightarrow{a} E'}$ with $H = \emptyset$. The case $H \neq \emptyset$ corresponds to the derivation of $E \xrightarrow{a} E'$ under the assumptions H .

²When $\mathcal{S}$ contains free variables from a set W , this solution is parameterised by the choice of a valuation of these variables as processes, thereby becoming a W -ary function.

Definition 4 (Proof) Let $\mathcal{T} = (\Sigma, \mathcal{R})$ be a TSS. A *proof* of a transition rule $\frac{H}{\alpha}$ from $\mathcal{T}$ is a well-founded, upwardly branching tree of which the nodes are labelled by Σ -literals, such that:

- the root is labelled by α , and
- if β is the label of a node q and K is the set of labels of the nodes directly above q , then
 - either $K = \emptyset$ and $\beta \in H$,
 - or $\frac{K}{\beta}$ is a substitution instance of a rule from $\mathcal{R}$.

If a proof of $\frac{H}{\alpha}$ from $\mathcal{T}$ exists, then $\frac{H}{\alpha}$ is *provable* from $\mathcal{T}$, notation $\mathcal{T} \vdash \frac{H}{\alpha}$.

A TSS is meant to specify an transition system in which the transitions are closed positive literals. A positive TSS specifies a transition relation in a straightforward way as the set of all provable transitions.³ But as pointed out in GROOTE [18], it is not so easy to associate a transition relation to a TSS with negative premises. In [15] several solutions to this problem were reviewed and evaluated. Arguably, the best method to assign a meaning to all TSSs is the *well-founded semantics* of VAN GELDER, ROSS & SCHLIPF [12], which in general yields a *3-valued transition relation* $T : \mathsf{T}(\Sigma) \times A \times \mathsf{T}(\Sigma) \rightarrow \{\text{present}, \text{undetermined}, \text{absent}\}$. I present such a relation as a pair $\langle CT, PT \rangle$ of 2-valued transition relations—the sets of *certain* and *possible transitions*—with $CT \subseteq PT$. When insisting on 2-valued transition relations, the best method is the same, declaring meaningful only those TSSs whose well-founded semantics is 2-valued, meaning that $CT = PT$. Such a TSS is called *complete* [15].

Below I follow the presentation from [16], which was strongly inspired by earlier accounts [24, 6, 15]. As Definition 4 does not allow the derivation of negative literals, to arrive at an approximation AT^+ of the set of transitions that are in the transition relation intended by a TSS $\mathcal{T}$, one could start from an approximation AT^- of the closed negative literals that ought to be generated, and define AT^+ as the set of closed positive literals provable from $\mathcal{T}$ under the hypotheses AT^- . Intuitively,

1. if AT^- is an under- (resp. over-)approximation of the closed negative literals that “really” hold, then AT^+ will be an under- (resp. over-)approximation of the intended (2-valued) transition relation, and
2. if AT^+ is an under- (resp. over-)approximation of the intended transition relation, then the set of all closed negative literals that do not deny any literal in AT^+ is an over- (resp. under-)approximation of the closed negative literals that agree with the intended transition relation.

Based on this insight, for each ordinal λ I define overapproximations PT_λ^+ and PT_λ^- of the closed positive and negative literals that the transition relation intends to generate, and underapproximations CT_λ^+ and CT_λ^- . The approximations get better with increasing λ . I start by taking the set of *all* closed negative literals as PT_0^- and then apply 1 and 2 above to generate these approximations in the order $PT_\lambda^- \rightarrow PT_\lambda^+ \rightarrow CT_\lambda^- \rightarrow CT_\lambda^+ \rightarrow PT_{\lambda+1}^- \rightarrow \dots$.

Definition 5 (Over- and underapproximations of transition relations [16]) Let $\mathcal{T}$ be a TSS. For ordinals λ the sets CT_λ^+ and PT_λ^+ of closed positive literals, and CT_λ^- , PT_λ^- of closed negative literals are defined inductively by:

$$\begin{array}{ll}
 PT_\lambda^- & \text{is the set of negative literals} \\
 & \text{that do not deny any} \\
 & \beta \in CT_\kappa^+ \text{ with } \kappa < \lambda \\
 CT_\lambda^- & \text{is the set of negative literals} \\
 & \text{that do not deny any } \beta \in PT_\lambda^+
 \end{array}
 \qquad
 \begin{array}{ll}
 \beta \in PT_\lambda^+ & \text{iff } P \vdash \frac{PT_\lambda^-}{\beta} \\
 \beta \in CT_\lambda^+ & \text{iff } P \vdash \frac{CT_\lambda^-}{\beta}.
 \end{array}$$

³Readers interested only in the restriction of my results to TSSs without negative premises—giving rise to 2-valued transition relations—can safely skip the remainder of this section, and identify $p \dashrightarrow p'$ with $p \xrightarrow{a} p'$. In the proof of Lemma 2 the induction on λ can be skipped, as well as Claims 1, 2 and 3, and the proof proceeds directly by induction on π .

Lemma 1 ([16]) $CT_\kappa^- \subseteq CT_\lambda^- \subseteq PT_\lambda^- \subseteq PT_\kappa^-$ and $CT_\kappa^+ \subseteq CT_\lambda^+ \subseteq PT_\lambda^+ \subseteq PT_\kappa^+$ for $\kappa < \lambda$.

Definition 6 (Well-founded semantics [16]) Define $PT^- := \bigcap_\lambda PT_\lambda^-$, taking the intersection over all ordinals. Likewise, $PT^+ := \bigcap_\lambda PT_\lambda^+$, $CT^- := \bigcup_\lambda CT_\lambda^-$ and $CT^+ := \bigcup_\lambda CT_\lambda^+$.

The 3-valued transition relation $\langle CT^+, PT^+ \rangle$ constitutes the well-founded semantics of a TSS.

Since the closed literals over Σ form a proper set, there must be an ordinal κ such that $PT_\lambda^- = PT_\kappa^-$ for all $\lambda > \kappa$, and hence also $PT_\lambda^+ = PT_\kappa^+$, $CT_\lambda^- = CT_\kappa^-$ and $CT_\lambda^+ = CT_\kappa^+$. Such an ordinal κ is called a *closure ordinal*. Now $PT^- = PT_\kappa^-$, $PT^+ = PT_\kappa^+$, $CT^- = CT_\kappa^-$ and $CT^+ = CT_\kappa^+$. Note that PT^- is the set of literals that do not deny any literal in CT^+ , and likewise for CT^- and PT^+ . Moreover, $CT^- \subseteq PT^-$ and $CT^+ \subseteq PT^+$. Furthermore $\beta \in PT^+$ iff $P \vdash \frac{PT^-}{\beta}$, and likewise $\beta \in CT^+$ iff $P \vdash \frac{CT^-}{\beta}$. In [16] it is shown that the above account of the well-founded semantics is consistent with the one in [15], and thereby with the ones in [6, 24, 12].

Below the statements $P \xrightarrow{a}_\lambda Q$, $P \not\xrightarrow{a}_\lambda Q$, $P \dashv\!\!\rightarrow_\lambda Q$ and $P \dashv\!\!\rightarrow_\lambda Q$ will abbreviate $(P \xrightarrow{a} Q) \in CT_\lambda^+$, $(P \not\xrightarrow{a}) \in CT_\lambda^-$, $(P \xrightarrow{a} Q) \in PT_\lambda^+$ and $P \dashv\!\!\rightarrow \in PT_\lambda^-$, respectively. Likewise, $P \xrightarrow{a} Q$ and $P \dashv\!\!\rightarrow Q$ abbreviate $(P \xrightarrow{a} Q) \in CT^+$, and $(P \dashv\!\!\rightarrow Q) \in PT^+$.

3 The ready simulation format with recursion

Definition 7 (Ready simulation format with recursion) An *ntyft/ntyxt rule* is a transition rule whose source contains at most one operator symbol and no multiple occurrences of the same variable, and in which the right-hand sides of positive premises are variables that are all distinct and do not occur in the source. It has *lookahead* if some variable both occurs in the right-hand side of a premise and in the left-hand side of a premise. A TSS is in the *ready simulation format with recursion* if for every recursive specification $\mathcal{S}$ and $X \in V_{\mathcal{S}}$ it has a rule

$$\frac{\langle \mathcal{S}_X | \mathcal{S} \rangle \xrightarrow{a} z}{\langle X | \mathcal{S} \rangle \xrightarrow{a} z}$$

with $z \in Var$ and all of its other rules are recursion-free⁴ ntyft/ntyxt rules without lookahead.

4 Guardedness

Definition 8 (Frozen arguments of operators [11]) Let Σ be a signature and Γ a unary predicate on the collection $\{(f, i) \mid 1 \leq i \leq ar(f), f \in \Sigma\}$ of arguments of operators. If $\Gamma(f, i)$, then argument i of f is $\bar{\Gamma}$ -frozen; otherwise it is $\bar{\Gamma}$ -liquid. An occurrence of a variable x in a term E is $\bar{\Gamma}$ -frozen iff E contains a subterm $f(E_1, \dots, E_{ar(f)})$ such that the occurrence of x is in E_i for a $\bar{\Gamma}$ -frozen argument i of f .

To prepare for the definition of guardedness below, I here use a predicate Γ that marks the frozen arguments of operators. However, I do not want to break the convention of [11], where a predicate Γ marks the liquid arguments. This is why I define frozen and liquid arguments and variable occurrences in terms of the complement $\bar{\Gamma}$ of my Γ .

Definition 9 Let $\mathcal{T}$ be a TSS in the ready simulation format with recursion. A predicate Γ as above *respects guardedness* for $\mathcal{T}$ if for each rule r of $\mathcal{T}$ with source $f(x_1, \dots, x_{ar(f)})$ and i a $\bar{\Gamma}$ -frozen argument of f , the variable x_i only occurs $\bar{\Gamma}$ -frozen in the left-hand sides of the premises of r .

⁴I conjecture that this restriction can be lifted at the expense of extra bookkeeping regarding bound variables. As I am not aware of any applications of such a generalisation I did not explore this further.

Note that the union of any collection of predicates Γ that respect guardedness for $\mathcal{T}$ is itself a predicate that respects guardedness for $\mathcal{T}$. Hence there exists a largest predicate $\Gamma_{\mathcal{T}}$ that respects guardedness for $\mathcal{T}$. Henceforth, $\mathcal{T}$ -guarded (or *guarded* when $\mathcal{T}$ is understood from context) will mean $\overline{\Gamma_{\mathcal{T}}}$ -frozen.

Thus, if r is a rule of $\mathcal{T}$ with source $f(x_1, \dots, x_{ar(f)})$ and i is a guarded ($= \overline{\Gamma_{\mathcal{T}}}$ -frozen) argument of f , then x_i only occurs guarded ($= \overline{\Gamma_{\mathcal{T}}}$ -frozen) in the left-hand sides of the premises of r .

Definition 10 Let $\mathcal{T}$ be a TSS in the ready simulation format with recursion. An occurrence of a variable x in a term E is $\mathcal{T}$ -guarded if it is $\overline{\Gamma}$ -frozen for some predicate Γ on arguments of operators that respects guardedness for $\mathcal{T}$. An term E is $\mathcal{T}$ -guarded if all free occurrences of variables in E are $\mathcal{T}$ -guarded.

The next lemma, that for CCS goes back to Milner [22], is a crucial tool in proving RSP. When creating or adapting a definition of guardedness, the main consideration is to make sure that this lemma holds. Below $\vec{P}, \vec{Q} \in T^W$ are W -tuples of processes; such tuples are also substitutions—hence the notation $G[\vec{P}]$.

Lemma 2 Let $G \in \mathbb{T}(\Sigma, W)$ be guarded and have free variables from $W \subseteq \text{Var}$ only, and let $\vec{P}, \vec{Q} \in T^W$. If $G[\vec{P}] \xrightarrow{a} R$ with $a \in A$, then R has the form $G'[\vec{P}]$ for some $G' \in \mathbb{T}(\Sigma, W)$. Moreover $G[\vec{Q}] \xrightarrow{a} G'[\vec{Q}]$. If $G[\vec{Q}] \not\xrightarrow{a} R$ with $a \in A$, then R has the form $G'[\vec{Q}]$ for some $G' \in \mathbb{T}(\Sigma, W)$. Moreover $G[\vec{P}] \not\xrightarrow{a} G'[\vec{P}]$.

Proof: Given $\vec{P}, \vec{Q} \in T^W$, I will show, for each ordinal λ , that for each guarded $G \in \mathbb{T}(\Sigma, W)$ and $a \in A$,

1. If $G[\vec{Q}] \not\xrightarrow{a/\lambda}$ then $G[\vec{P}] \not\xrightarrow{a/\lambda}$,
2. If $G[\vec{Q}] \xrightarrow{a/\lambda} R$ then R has the form $G'[\vec{Q}]$ for some $G' \in \mathbb{T}(\Sigma, W)$. Moreover $G[\vec{P}] \xrightarrow{a/\lambda} G'[\vec{P}]$,
3. If $G[\vec{P}] \not\xrightarrow{a/\lambda}$ then $G[\vec{Q}] \not\xrightarrow{a/\lambda}$, and
4. If $G[\vec{P}] \xrightarrow{a/\lambda} R$ then R has the form $G'[\vec{P}]$ for some $G' \in \mathbb{T}(\Sigma, W)$. Moreover $G[\vec{Q}] \xrightarrow{a/\lambda} G'[\vec{Q}]$.

The statements of the lemma then follow by taking λ to be a closure ordinal. I apply induction on λ .

1. Suppose $G[\vec{P}] \not\xrightarrow{a/\lambda}$ does not hold. Then, by Definition 5, the literal $G[\vec{P}] \not\xrightarrow{a/\lambda}$ denies a $\beta \in CT_{\kappa}^+$ with $\kappa < \lambda$. The literal β must have the form $G[\vec{P}] \xrightarrow{a/\kappa} R$. By induction, using Claim 4 above, $G[\vec{Q}] \xrightarrow{a/\kappa} R'$ for some $R' \in T(\Sigma)$. Consequently, $G[\vec{Q}] \not\xrightarrow{a/\lambda}$ does not hold.
2. The proof of Claim 2 proceeds exactly like the one of Claim 4 below.
3. Suppose $G[\vec{Q}] \not\xrightarrow{a/\lambda}$ does not hold. Then, by Definition 5, the literal $G[\vec{Q}] \not\xrightarrow{a/\lambda}$ denies a $\beta \in PT_{\lambda}^+$. The literal β must have the form $G[\vec{Q}] \xrightarrow{a/\lambda} R$. Using Claim 2 above, $G[\vec{Q}] \xrightarrow{a/\lambda} R'$ for some $R' \in T(\Sigma)$. Consequently, $G[\vec{P}] \not\xrightarrow{a/\lambda}$ does not hold.
4. The proof of this claim proceeds by a nested induction on the derivation π of $G[\vec{P}] \xrightarrow{a} R$ from the hypotheses CT_{λ}^- , making a case distinction on the shape of G . Trivially, if a closed positive literal is provable from a set of closed literals, all literals occurring in the proof can be chosen closed. I assume this is the case for π .

The case that $G = x \in \text{Var}$ cannot occur, as G is guarded.

Let $G = f(G_1, \dots, G_{ar(f)})$ for some $f \in \Sigma$ and $G_1, \dots, G_{ar(f)} \in \mathbb{T}(\Sigma, W)$. Let $r = \frac{H}{f(x_1, \dots, x_{ar(f)}) \xrightarrow{a} F}$ be the ntyft/ntyxt-rule without lookahead and $\sigma : V \rightarrow T(\Sigma)$ the substitution that are used in the last step of π . Here V is the set of free variables occurring in r . Then $F[\sigma] = R$ and $f(x_1, \dots, x_{ar(f)})[\sigma] = G[\vec{P}]$, so $\sigma(x_i) = G_i[\vec{P}]$ for $i = 1, \dots, ar(f)$. Now define the substitution $\rho : V \rightarrow \mathbb{T}(\Sigma, W)$ by

$$\begin{aligned} \rho(x_i) &= G_i && \text{for } i = 1, \dots, ar(f), \text{ and} \\ \rho(y) &= \sigma(y) && \text{for all } y \in V \setminus \{x_1, \dots, x_{ar(f)}\}. \end{aligned}$$

Then $E[\sigma] = E[\rho][\vec{P}]$ for any recursion-free term $E \in \mathbb{T}(\Sigma, V)$.

Given a positive premise $E_y \xrightarrow{c} y$ in H , one finds that $E_y[\rho][\vec{P}] = E_y[\sigma] \xrightarrow{c/\lambda} \sigma(y)$, and the proof of this transition is a subproof of π . Consequently, I may apply the induction hypothesis on this transition, provided I can show that $E_y[\rho]$ is guarded.

Any free occurrence of a variable y in $E_y[\rho]$ must lay in a term G_i , for some $i \in \{1, \dots, ar(f)\}$, that is substituted by ρ for an occurrence of x_i in E_y . In case x_i is a guarded argument of f , each occurrence of x_i in E_y is guarded, and hence the occurrence of y is guarded in $E_y[\rho]$ as well. In case x_i is an unguarded argument of f , as this occurrence of y must be guarded in G , it is guarded in G_i , and thus also in $E_y[\rho]$.

So $E_y[\rho]$ is guarded indeed. By induction, $\sigma(y)$ has the form $E'_y[\vec{P}]$ for some $E'_y \in \mathbb{T}(\Sigma, W)$. Moreover $E_y[\rho][\vec{Q}] \xrightarrow{c}_\lambda E'_y[\vec{Q}]$. Let V_{RHS} be the set of variables occurring as right-hand sides of premises in H , and define the substitution $\bar{\rho} : V \rightarrow \mathbb{T}(\Sigma, W)$ by

$$\begin{aligned} \bar{\rho}(x_i) &= G_i && \text{for } i = 1, \dots, ar(f), \\ \bar{\rho}(y) &= E'_y && \text{for } y \in V_{\text{RHS}}, \text{ and} \\ \bar{\rho}(y) &= \sigma(y) && \text{for all } y \in V \setminus (\{x_1, \dots, x_{ar(f)}\} \cup V_{\text{RHS}}). \end{aligned}$$

Then $E[\sigma] = E[\bar{\rho}][\vec{P}]$ for any recursion-free term $E \in \mathbb{T}(\Sigma, V)$. Thus $R = F[\sigma] = F[\bar{\rho}][\vec{P}]$, which is the first statement that needed to be proved.

Towards the second, let $\bar{\sigma} : V \rightarrow \mathbb{T}(\Sigma)$ be the substitution given by

$$\begin{aligned} \bar{\sigma}(x_i) &= G_i[\vec{Q}] && \text{for } i = 1, \dots, ar(f), \\ \bar{\sigma}(y) &= E'_y[\vec{Q}] && \text{for } y \in V_{\text{RHS}}, \text{ and} \\ \bar{\sigma}(y) &= \sigma(y) && \text{for all } y \in V \setminus (\{x_1, \dots, x_{ar(f)}\} \cup V_{\text{RHS}}). \end{aligned}$$

Then $E[\bar{\sigma}] = E[\bar{\rho}][\vec{Q}]$ for any recursion-free term $E \in \mathbb{T}(\Sigma, V)$. For each premise $E_y \xrightarrow{c}_\lambda y$ in H one has $E_y[\rho] = E_y[\bar{\rho}]$ since the rule r has no lookahead. Thus

$$E_y[\bar{\sigma}] = E_y[\bar{\rho}][\vec{Q}] = E_y[\rho][\vec{Q}] \xrightarrow{c}_\lambda E'_y[\vec{Q}] = y[\bar{\sigma}].$$

For each premise $E \xrightarrow{c}_\lambda$ in H one has $E[\rho][\vec{P}] = E[\sigma] \xrightarrow{c}_\lambda$, and $E[\rho]$ is guarded by the same reasoning as above. By Claim 3 above, $E[\rho][\vec{Q}] \xrightarrow{c}_\lambda$. Again $E[\rho] = E[\bar{\rho}]$, as r has no lookahead, so $E[\bar{\sigma}] = E[\bar{\rho}][\vec{Q}] = E[\rho][\vec{Q}] \xrightarrow{c}_\lambda$. Applying rule r , it follows that

$$G[\vec{Q}] = f(x_1, \dots, x_{ar(f)})[\bar{\sigma}] \xrightarrow{a}_\lambda F[\bar{\sigma}] = F[\bar{\rho}][\vec{Q}],$$

which had to be shown.

Finally, let $G = \langle X | \mathcal{S} \rangle$, so that $G[\vec{P}] = \langle X | \mathcal{S}[\vec{P}^\dagger] \rangle$, where $\vec{P}^\dagger$ is the $W \setminus V_S$ -tuple that is left of $\vec{P}$ after deleting the Y -components, for $Y \in V_S$. The transition $\langle \mathcal{S}_X[\vec{P}^\dagger] | \mathcal{S}[\vec{P}^\dagger] \rangle \xrightarrow{a}_\lambda R$ is derivable through a subderivation of the one for $\langle X | \mathcal{S}[\vec{P}^\dagger] \rangle \xrightarrow{a}_\lambda R$. Moreover, $\langle \mathcal{S}_X[\vec{P}^\dagger] | \mathcal{S}[\vec{P}^\dagger] \rangle = \langle \mathcal{S}_X | \mathcal{S} \rangle[\vec{P}]$. So by induction, R has the form $E[\vec{P}]$ for some $E \in \mathbb{T}(\Sigma, W)$, and $\langle \mathcal{S}_X | \mathcal{S} \rangle[\vec{Q}] \xrightarrow{a}_\lambda E[\vec{Q}]$. Since $\langle \mathcal{S}_X | \mathcal{S} \rangle[\vec{Q}] = \langle \mathcal{S}_X[\vec{Q}^\dagger] | \mathcal{S}[\vec{Q}^\dagger] \rangle$, it follows that $G[\vec{Q}] = \langle X | \mathcal{S}[\vec{Q}] \rangle = \langle X | \mathcal{S}[\vec{Q}^\dagger] \rangle \xrightarrow{a}_\lambda G'[\vec{Q}]$. $\square$

Definition 11 (Guarded recursive specifications) A recursive specification $\mathcal{S}$ is *manifestly guarded* if for each $X, Y \in V_S$ all occurrences of the variable Y in the term $\mathcal{S}_X$ are guarded. It is *guarded* if it can be converted into a manifestly guarded recursive specification by repeated substitution of expressions $\mathcal{S}_Y$ for variables $Y \in V_S$ occurring in the expressions $\mathcal{S}_Z$ for $Z \in V_S$.

5 The bisimulation preorder

Traditionally [22], bisimilarity is defined on 2-valued transition systems only, whereas the structural operational semantics of a language specified by a TSS can be 3-valued. Rather than limit my results to languages specified by complete TSSs, I follow [16] in using an extension of the notion of bisimilarity to 3-valued transition systems. This extension stems from [20] under the name *modal refinement*; there 3-valued transition systems are called *modal transition systems*.

Definition 12 (Bisimilarity) Let $\mathcal{T} = (\Sigma, \mathcal{R})$ be a TSS. A *bisimulation* $\mathcal{B}$ is a binary relation on $T(\Sigma)$ such that, for $P, Q \in T(\Sigma)$ and $a \in A$,

- if $P \mathcal{B} Q$ and $P \xrightarrow{a} P'$, then there is a Q' with $Q \xrightarrow{a} Q'$ and $P' \mathcal{B} Q'$,
- if $P \mathcal{B} Q$ and $Q \xrightarrow{a} Q'$, then there is a P' with $P \xrightarrow{a} P'$ and $P' \mathcal{B} Q'$.

A process $Q \in T(\Sigma)$ is a *modal refinement* of $P \in T(\Sigma)$, notation $P \sqsubseteq_B Q$, if there exists a bisimulation $\mathcal{B}$ with $P \mathcal{B} Q$. I call $\sqsubseteq_B$ the *bisimulation preorder*, or *bisimilarity*. The kernel of $\sqsubseteq$, given by $\equiv_B := \sqsubseteq_B \cap \sqsupseteq_B$, is *bisimulation equivalence*.

Clearly, modal refinement is reflexive and transitive, and hence a preorder. The underlying idea is that a process P with a 3-valued transition relation $\langle CT, PT \rangle$ is a *specification* of a process with a 2-valued transition relation, in which the presence or absence of certain transitions is left open. CT contains the transitions that are *required* by the specification, and PT the ones that are *allowed*. If $P \sqsubseteq_B Q$, then Q may be closer to the eventual implementation, in the sense that some of the undetermined transitions have been resolved to present or absent. The requirements of Definition 12 now say that any transition that is required by P should be (matched by a transition) required by Q , whereas any transition allowed by Q , should certainly be (matched by a transition) allowed by P .

In case P and Q are 2-valued (i.e. *implementations*) the modal refinement relation is just the traditional notion of bisimilarity [22] (and thus symmetric).

Theorem 1 (Lean congruence [16]) Let $\mathcal{T} = (\Sigma, \mathcal{R})$ be a TSS in the ready simulation format with recursion. Let $E \in \mathbb{T}(\Sigma, W)$ be an expression, and let $\vec{P}, \vec{Q} \in T(\Sigma)^W$ be W -tuples of processes for some $W \subseteq \text{Var}$, such that $\vec{P}(x) \sqsubseteq_B \vec{Q}(x)$ for each $x \in W$. Then $E[\vec{P}] \sqsubseteq_B E[\vec{Q}]$.

In [16] this result was actually obtained for the more general ntyft/ntyxt-format with recursion, obtained by lifting the restriction ruling out lookahead. The special case that E is a recursion-free expression stems from [6] (under a side condition on the TSS called *well-foundedness*, which was lifted in [10]). The special subcase in which $\mathcal{T}$ has no negative premises goes back to [19].

Milner [22] proposed an important tool in proving bisimilarity between two processes: the notion of a bisimulation up to bisimilarity. Here I extend it to the bisimulation preorder on 3-valued TSSs.

Definition 13 (Bisimulation upto) Let $\mathcal{T} = (\Sigma, \mathcal{R})$ be a TSS. A *bisimulation $\mathcal{B}$ up to bisimilarity* is a binary relation on $T(\Sigma)$ such that, for $P, Q \in T(\Sigma)$ and $a \in A$,

- if $P \mathcal{B} Q$ and $P \xrightarrow{a} P'$, then there is a Q' with $Q \xrightarrow{a} Q'$ and $P' \sqsubseteq_B \mathcal{B} \sqsubseteq_B Q'$,
- if $P \mathcal{B} Q$ and $Q \xrightarrow{a} Q'$, then there is a P' with $P \xrightarrow{a} P'$ and $P' \sqsubseteq_B \mathcal{B} \sqsubseteq_B Q'$.

Here $\sqsubseteq_B \mathcal{B} \sqsubseteq_B := \{(R, S) \mid \exists R', S'. R \sqsubseteq_B R' \mathcal{B} S' \sqsubseteq_B S\}$.

Proposition 1 If $P \mathcal{B} Q$ for some bisimulation $\mathcal{B}$ up to bisimilarity, then $P \sqsubseteq_B Q$.

Proof: Using the reflexivity of $\sqsubseteq_B$ it suffices to show that $\sqsubseteq_B \mathcal{B} \sqsubseteq_B$ is a bisimulation. Using transitivity of $\sqsubseteq_B$ this is straightforward. $\square$

6 The recursive specification principle

Let Σ be a signature. Any preorder $\sqsubseteq \subseteq T(\Sigma) \times T(\Sigma)$ on the closed terms over Σ extends to a preorder $\sqsubseteq \subseteq \mathbb{T}(\Sigma) \times \mathbb{T}(\Sigma)$ on open terms by defining $E \sqsubseteq F$ iff $E[\rho] \sqsubseteq F[\rho]$ for each closed substitution ρ . It extends to substitutions $\rho, \nu : \text{Var} \rightarrow \mathbb{T}(\Sigma)$ by $\rho \sim \nu$ iff $\text{dom}(\rho) = \text{dom}(\nu)$ and $\rho(x) \sim \nu(x)$ for each $x \in \text{dom}(\rho)$. As W -tuples $\vec{E}, \vec{F} \in \mathbb{T}(\Sigma)^W$ for $W \subseteq \text{Var}$ are substitutions, this gives $\vec{E} \sqsubseteq \vec{F}$ iff $\vec{E}(x) \sqsubseteq \vec{F}(x)$ for each $x \in W$. A recursive specification $S : V_S \rightarrow \mathbb{T}(\Sigma)$ is a V_S -tuple; if $\vec{E} \in \mathbb{T}(\Sigma)^{V_S}$ is another V_S -tuple then $S[\vec{E}]$ I denote the V_S -tuple with elements $S_X[\vec{E}]$ for each $X \in V_S$, that is, the body S_X of the recursive equation of X in which $\vec{E}(Y)$ is substituted for each variable $Y \in V_S$ occurring freely in S_X .

Definition 14 (Solution) For $\mathcal{S}$ a recursive specification and $\equiv$ a semantic equivalence, a V_S -tuple of expressions $\vec{E} \in \mathbb{T}(\Sigma)^{V_S}$ is a *solution* of $\mathcal{S}$ if $\vec{E} \equiv \mathcal{S}[\vec{E}]$.

When generalising this definition with a preorder $\sqsubseteq$ in the rôle of $\equiv$, one obtains *pre-* and *postsolutions*.

Definition 15 For $\mathcal{S}$ a recursive specification and $\sqsubseteq$ a semantic equivalence, a V_S -tuple of expressions $\vec{E} \in \mathbb{T}(\Sigma)^{V_S}$ is a *presolution* (resp. *postsolution*) of $\mathcal{S}$ if $\vec{E} \sqsubseteq \mathcal{S}[\vec{E}]$ (resp. $\mathcal{S}[\vec{E}] \sqsubseteq \vec{E}$).

The following theorem establishes RSP for the bisimulation preorder.

Theorem 2 Let $\mathcal{T} = (\Sigma, \mathcal{R})$ be a TSS in the ready simulation format with recursion, and $\mathcal{S}$ be a $\mathcal{T}$ -guarded recursive specification. If $\vec{E} \sqsubseteq_B \mathcal{S}[\vec{E}]$ and $\mathcal{S}[\vec{F}] \sqsubseteq_B \vec{F}$ with $\vec{E}, \vec{F} \in \mathbb{T}(\Sigma)^{V_S}$, then $\vec{E} \sqsubseteq_B \vec{F}$.

Proof: It suffices to prove Theorem 2 under the assumptions that $\vec{E}, \vec{F} \in \mathbb{T}(\Sigma)^{V_S}$ and only the variables from V_S occur free in the expressions S_X for $X \in V_S$. For in the general case I have to establish that $\vec{E}[\sigma] \sqsubseteq_B \vec{F}[\sigma]$ for an arbitrary closed substitution $\sigma : \text{Var} \rightarrow \mathbb{T}(\Sigma)$. Let $\sigma^\dagger : \text{Var} \setminus V_S \rightarrow \mathbb{T}(\Sigma)$ be given by $\sigma^\dagger(X) = \sigma(X)$ for all $X \in \text{Var} \setminus V_S$. Then $\vec{E} \sqsubseteq_B \mathcal{S}[\vec{E}]$ and $\mathcal{S}[\vec{F}] \sqsubseteq_B \vec{F}$ imply $\vec{E}[\sigma] \sqsubseteq_B \mathcal{S}[\vec{E}][\sigma] = \mathcal{S}[\sigma^\dagger][\vec{E}[\sigma]]$ and $\mathcal{S}[\sigma^\dagger][\vec{F}[\sigma]] = \mathcal{S}[\vec{F}][\sigma] \sqsubseteq_B \vec{F}[\sigma]$. Hence, I merely have to prove the theorem with $\vec{E}[\sigma]$, $\vec{F}[\sigma]$ and $\mathcal{S}[\sigma^\dagger]$ in place of $\vec{E}$, $\vec{F}$ and $\mathcal{S}$.

It also suffices to prove Theorem 2 under the assumption that $\mathcal{S}$ is a manifestly guarded recursive specification. Namely, for a general guarded recursive specification $\mathcal{S}$, let $\mathcal{S}'$ be the manifestly guarded specification into which $\mathcal{S}$ can be converted. Then $\vec{E} \sqsubseteq_B \mathcal{S}[\vec{E}]$ implies $\vec{E} \sqsubseteq_B \mathcal{S}'[\vec{E}]$ by Theorem 1 and transitivity, and likewise $\mathcal{S}[\vec{F}] \sqsubseteq_B \vec{F}$ implies $\mathcal{S}'[\vec{F}] \sqsubseteq_B \vec{F}$.

So let $\mathcal{S}$ be manifestly guarded with free variable from V_S only, and let $\vec{P}, \vec{Q} \in \mathbb{T}^{V_S}(\Sigma)$ be a pre- and postsolution of $\mathcal{S}$, that is, $\vec{P} \sqsubseteq_B \mathcal{S}[\vec{P}]$ and $\mathcal{S}[\vec{Q}] \sqsubseteq_B \vec{Q}$. I will show that

$$\mathcal{B} := \{G[\mathcal{S}[\vec{P}]], G[\mathcal{S}[\vec{Q}]] \mid G \in \mathbb{T}(\Sigma) \text{ is recursion-free and has variables from } V_S \text{ only}\}$$

is a bisimulation up to bisimilarity. Once I have that, taking $G := X \in V_S$ yields $S_X[\vec{P}] \sqsubseteq_B S_X[\vec{Q}]$ by Proposition 1, and thus $\vec{P}(X) \sqsubseteq_B S_X[\vec{P}] \sqsubseteq_B S_X[\vec{Q}] \sqsubseteq_B \vec{Q}(X)$ for all $X \in V_S$. So $\vec{P} \sqsubseteq_B \vec{Q}$.

- Let $R \mathcal{B} T$ and $R \xrightarrow{a} R'$ with $a \in A$. I have to find a T' with $T \xrightarrow{a} T'$ and $R' \sqsubseteq_B \mathcal{B} \sqsubseteq_B T'$. Let $R = G[\mathcal{S}[\vec{P}]]$ and $T = G[\mathcal{S}[\vec{Q}]]$, where $G \in \mathbb{T}(\Sigma)$ is recursion-free and has variables from V_S only. Note that $G[\mathcal{S}[\vec{P}]]$ can also be written as $G[\mathcal{S}][\vec{P}]$. Since the expressions S_X for $X \in V_S$ have free variables from V_S only, so does $G[\mathcal{S}]$. Moreover, since $\mathcal{S}$ is manifestly guarded, the expression $G[\mathcal{S}]$ must be guarded. By Lemma 2, R' must have the form $G'[\vec{P}]$, where $G' \in \mathbb{T}(\Sigma, V_S)$. Moreover, $T = G[\mathcal{S}[\vec{Q}]] = G[\mathcal{S}][\vec{Q}] \xrightarrow{a} G'[\vec{Q}] =: T'$. Furthermore, by Theorem 1, $G'[\vec{P}] \sqsubseteq_B G'[\mathcal{S}[\vec{P}]]$ and $G'[\mathcal{S}[\vec{Q}]] \sqsubseteq_B G'[\vec{Q}]$. By definition $G'[\mathcal{S}[\vec{P}]] \mathcal{B} G'[\mathcal{S}[\vec{Q}]]$. Thus, $R' = G'[\vec{P}] \sqsubseteq_B \mathcal{B} \sqsubseteq_B G'[\vec{Q}] = T'$.
- Let $R \mathcal{B} T$ and $T \xrightarrow{a} T'$ with $a \in A$. I have to find an R' with $R \xrightarrow{a} R'$ and $R' \sqsubseteq_B \mathcal{B} \sqsubseteq_B T'$. Let $R = G[\mathcal{S}[\vec{P}]]$ and $T = G[\mathcal{S}[\vec{Q}]]$, where $G \in \mathbb{T}(\Sigma)$ is recursion-free and has variables from V_S only. Note that $G[\mathcal{S}[\vec{Q}]]$ can also be written as $G[\mathcal{S}][\vec{Q}]$. Since the expressions S_X for $X \in V_S$ have free variables from V_S only, so does $G[\mathcal{S}]$. Moreover, since $\mathcal{S}$ is manifestly guarded, the expression $G[\mathcal{S}]$ must be guarded. By Lemma 2, T' must have the form $G'[\vec{Q}]$, where $G' \in \mathbb{T}(\Sigma, V_S)$. Moreover, $R = G[\mathcal{S}[\vec{P}]] = G[\mathcal{S}][\vec{P}] \xrightarrow{a} G'[\vec{P}] =: R'$. Furthermore, by Theorem 1, $G'[\vec{P}] \sqsubseteq_B G'[\mathcal{S}[\vec{P}]]$ and $G'[\mathcal{S}[\vec{Q}]] \sqsubseteq_B G'[\vec{Q}]$. By definition $G'[\mathcal{S}[\vec{P}]] \mathcal{B} G'[\mathcal{S}[\vec{Q}]]$. Thus, $R' = G'[\vec{P}] \sqsubseteq_B \mathcal{B} \sqsubseteq_B G'[\vec{Q}] = T'$. $\square$

The recursive definition principle, saying that recursive specifications have a solution, trivially holds for bisimulation equivalence, and thereby also for all coarser equivalences. Namely the recursive call $\langle X | \mathcal{S} \rangle$ itself is a solution of the recursive specification $\mathcal{S}$.

$$\langle X | \mathcal{S} \rangle \equiv_B \langle S_X | \mathcal{S} \rangle \quad \text{for all } X \in V_S$$

This follows immediately from the operational rules for recursion. In view of that, writing $\langle S|S \rangle$ for the V_S -tuple with elements $\langle S_X|S \rangle$ for $X \in V_S$, RSP can be restated as

$$\frac{\vec{P} \equiv_B \mathcal{S}[\vec{P}]}{\vec{P} \equiv_B \langle \mathcal{S}|S \rangle} \quad \frac{\vec{P} \sqsubseteq_B \mathcal{S}[\vec{P}]}{\vec{P} \sqsubseteq_B \langle \mathcal{S}|S \rangle} \quad \frac{\mathcal{S}[\vec{Q}] \sqsubseteq_B \vec{Q}}{\langle \mathcal{S}|S \rangle \sqsubseteq_B \vec{Q}}.$$

The ready simulation preorder $\sqsubseteq_{RS}$ [5, 14] has been generalised to 3-valued transition systems in [4]. Its definition is just like Definition 12, but without the targets Q' and P' in the second clause. Here $Q \xrightarrow{a}$ means that there exists a Q' such that $Q \xrightarrow{a} Q'$. In [4] it is also shown to be a congruence (cf. Theorem 1) on TSSs in the ready simulation format, albeit without recursion. Using this, the proof of Theorem 2 generalises smoothly to $\sqsubseteq_{RS}$. There merely is a straightforward simplification in the second clause, dealing with transitions $\xrightarrow{\quad}$. I conclude that RSP holds for the ready simulation preorder for TSSs in the ready simulation format.

The simulation preorder [15] is a congruence only for TSSs in the positive ready simulation format, that is, ruling out negative premises. It is defined just like the bisimulation preorder, but without its second clause. Again a straightforward simplification of the above proof shows that RSP holds for the simulation preorder for TSSs in the ready simulation format.

7 Future work

It would be great to extend this work to branching bisimilarity and/or other weak equivalences.

References

- [1] J.C.M. Baeten, J.A. Bergstra & J.W. Klop (1987): *On the consistency of Koomen's fair abstraction rule*. *Theoretical Computer Science* 51(1/2), pp. 129–176, doi:10.1016/0304-3975(87)90052-1.
- [2] J.C.M. Baeten & W.P. Weijland (1990): *Process Algebra*. Cambridge Tracts in Theoretical Computer Science 18, Cambridge University Press, doi:10.1017/CBO9780511624193.
- [3] J.A. Bergstra & J.W. Klop (1988): *A complete inference system for regular processes with silent moves*. In F.R. Drake & J.K. Truss, editors: *Proceedings Logic Colloquium 1986* Hull, North-Holland, pp. 21–81. First appeared as: Report CS-R8420, CWI, Amsterdam, 1984.
- [4] B. Bloom, W.J. Fokkink & R.J. van Glabbeek (2004): *Precongruence Formats for Decorated Trace Semantics*. *Transactions on Computational Logic* 5(1), pp. 26–78, doi:10.1145/963927.963929. Available at <http://theory.stanford.edu/~rvg/abstracts.html#48>.
- [5] B. Bloom, S. Istrail & A.R. Meyer (1995): *Bisimulation Can't be Traced*. *Journal of the ACM* 42(1), pp. 232–268, doi:10.1145/200836.200876.
- [6] R.N. Bol & J.F. Groote (1996): *The meaning of negative premises in transition system specifications*. *Journal of the ACM* 43(5), pp. 863–914, doi:10.1145/234752.234756.
- [7] E. Bres, R.J. van Glabbeek & P. Höfner (2016): *A Timed Process Algebra for Wireless Networks with an Application in Routing*. Technical Report 9145, NICTA. Available at <http://arxiv.org/abs/1606.03663>. Extended abstract in P. Thiemann, editor: *Programming Languages and Systems: Proceedings 25th European Symposium on Programming, ESOP'16*; held as part of the *European Joint Conferences on Theory and Practice of Software, ETAPS'16*, Eindhoven, The Netherlands, April 2016, LNCS 9632, Springer, 2016, pp. 95–122, doi:10.1007/978-3-662-49498-1_5.
- [8] S.D. Brookes, C.A.R. Hoare & A.W. Roscoe (1984): *A theory of communicating sequential processes*. *Journal of the ACM* 31(3), pp. 560–599, doi:10.1145/828.833.

- [9] W. J. Fokkink (2000): *Introduction to Process Algebra*. Texts in Theoretical Computer Science, An EATCS Series, Springer, doi:10.1007/978-3-662-04293-9.
- [10] W.J. Fokkink & R.J. van Glabbeek (1996): *Ntyft/ntyxt rules reduce to ntree rules*. *Information and Computation* 126(1), pp. 1–10, doi:10.1006/inco.1996.0030. Available at <http://theory.stanford.edu/~rvg/abstracts.html#33>.
- [11] W.J. Fokkink, R.J. van Glabbeek & P. de Wind (2012): *Divide and congruence: From decomposition of modal formulas to preservation of branching and η -bisimilarity*. *Information and Computation* 214, pp. 59–85, doi:10.1016/j.ic.2011.10.011.
- [12] A. van Gelder, K. Ross & J.S. Schlipf (1991): *The well-founded semantics for general logic programs*. *Journal of the ACM* 38(3), pp. 620–650, doi:10.1145/116825.116838.
- [13] R.J. van Glabbeek (1993): *Full Abstraction in Structural Operational Semantics (extended abstract)*. In M. Nivat, C. Rattray, T. Rus & G. Scollo, editors: *Proceedings of the 3rd International Conference on Algebraic Methodology and Software Technology, AMAST’93*, Twente, The Netherlands, June 1993, Workshops in Computing, Springer, pp. 75–82. Available at <http://theory.stanford.edu/~rvg/abstracts.html#28>.
- [14] R.J. van Glabbeek (2001): *The Linear Time – Branching Time Spectrum I: The Semantics of Concrete, Sequential Processes*. In J.A. Bergstra, A. Ponse & S.A. Smolka, editors: *Handbook of Process Algebra*, chapter 1, Elsevier, pp. 3–99, doi:10.1016/B978-044482830-9/50019-9.
- [15] R.J. van Glabbeek (2004): *The Meaning of Negative Premises in Transition System Specifications II*. *Journal of Logic and Algebraic Programming* 60–61, pp. 229–258, doi:10.1016/j.jlap.2004.03.007.
- [16] R.J. van Glabbeek (2017): *Lean and Full Congruence Formats for Recursion*. In: *Proceedings 32nd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS’17*, Reykjavik, Iceland, June 2017, IEEE Computer Society Press, doi:10.1109/LICS.2017.8005142.
- [17] R.J. van Glabbeek & C.A. Middelburg (2020): *On Infinite Guarded Recursive Specifications in Process Algebra*. Available at <http://arxiv.org/abs/2005.00746>.
- [18] J.F. Groote (1993): *Transition System Specifications with Negative Premises*. *Theoretical Computer Science* 118, pp. 263–299, doi:10.1016/0304-3975(93)90111-6.
- [19] J.F. Groote & F.W. Vaandrager (1992): *Structured Operational Semantics and Bisimulation as a Congruence*. *Information and Computation* 100(2), pp. 202–260, doi:10.1016/0890-5401(92)90013-6.
- [20] K.G. Larsen & B. Thomsen (1988): *A Modal Process Logic*. In: *Proc. LICS’88*, IEEE Computer Society Press, pp. 203–210, doi:10.1109/LICS.1988.5119.
- [21] R. Milner (1984): *A complete inference system for a class of regular behaviours*. *Journal of Computer and System Sciences* 28, pp. 439–466, doi:10.1016/0022-0000(84)90023-0.
- [22] R. Milner (1990): *Operational and algebraic semantics of concurrent processes*. In J. van Leeuwen, editor: *Handbook of Theoretical Computer Science*, chapter 19, Elsevier Science Publishers B.V. (North-Holland), pp. 1201–1242. Alternatively see *Communication and Concurrency*, Prentice-Hall, Englewood Cliffs, 1989, of which an earlier version appeared as *A Calculus of Communicating Systems*, LNCS 92, Springer, 1980, doi:10.1007/3-540-10235-3.
- [23] G.D. Plotkin (2004): *A Structural Approach to Operational Semantics*. *The Journal of Logic and Algebraic Programming* 60–61, pp. 17–139, doi:10.1016/j.jlap.2004.05.001. Originally appeared in 1981.
- [24] T.C. Przymusiński (1990): *The Well-founded Semantics Coincides with the Three-valued Stable Semantics*. *Fundamenta Informaticae* XIII(4), pp. 445–463.
- [25] A.W. Roscoe (1997): *The Theory and Practice of Concurrency*. Prentice-Hall. Available at <http://www.comlab.ox.ac.uk/bill.roscoe/publications/68b.pdf>.
- [26] F.W. Vaandrager (1993): *Expressiveness Results for Process Algebras*. In J.W. de Bakker, W.P. de Roever & G. Rozenberg, editors: *Proceedings REX Workshop on Semantics: Foundations and Applications*, Beekbergen, The Netherlands, 1992, LNCS 666, Springer, pp. 609–638, doi:10.1007/3-540-56596-5_49.